Lecture 8. Randomized Computation and Primality Test

Spring 2026, *Theory of Computation*, IIIS, Tsinghua

Why Study Probabilistic TM?

- To attempt to answer the question:
  - Does randomness add power?
  - Or putting it another way:

    Are there problems that can be solved by a probabilistic TM (in polynomial time) but these same problems cannot be solved by a deterministic TM in polynomial time?



Example– additive spanner

- A spanner S in G is a spanning subgraph of G which satisfies two crucial properties:
  1. sparse
  2. preserves approximate distances pair-wise



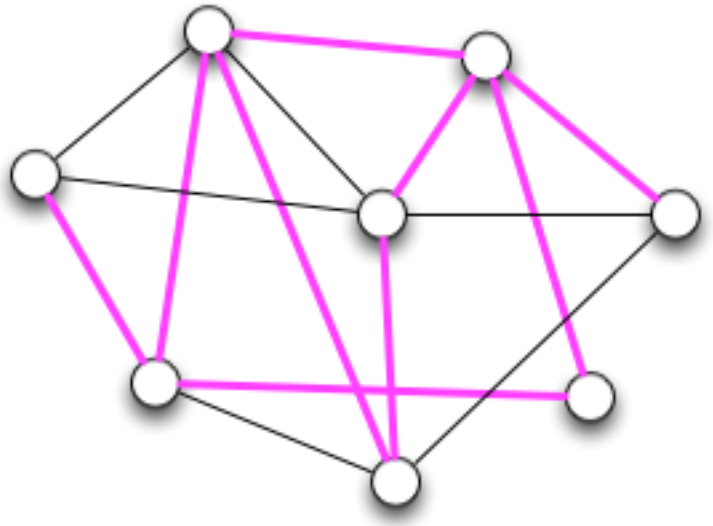
Example– additive spanner

- A spanner S in G is a spanning subgraph of G which satisfies two crucial properties:
  1. sparse
  2. preserves approximate distances pair-wise

- it's a generalization of spanning tree!
- in a spanning tree T, $d_T(u,v)<\infty$ iff $d_G(u,v)<\infty$



- Note that a spanner is a subgraph of G, so the distances in it can only increase
  - We hope they don't increase a lot!





Example– additive spanner

- A spanner S in G is a spanning subgraph of G which satisfies two crucial properties:
  1. sparse
  2. preserves approximate distances pair-wise

- Additive spanner: for all u,v∈G, $d_S(u,v) \leq d_G(u,v)+c$
  - $d_G(u,v)$ is the distance between u and v in G
  - c is a constant

- We consider the additive spanner for undirected unweighted graphs



2-additive spanner with $O(n^{3/2})$ edges

- The construction of the spanner S:

S=∅
Randomly select $n^{1/2}$ vertices, call the set W
S includes the shortest path trees from all vertices in W
for all vertices u:
    if there is an edge (u,w) s.t. w∈W,
        add (u,w) in S
    else include all edges (u,v) in S



2-additive spanner with $O(n^{3/2})$ edges

- The construction of the spanner S:

S=∅
Randomly select $n^{1/2}$ vertices, call the set W
S includes the shortest path trees from all vertices in W
for all vertices u:
if there is an edge (u,w) s.t. w∈W,
add (u,w) in S
else include all edges (u,v) in S

#edges is $O(n \cdot n^{1/2})$



2-additive spanner with $O(n^{3/2})$ edges

- The construction of the spanner S:

S=∅
Randomly select $n^{1/2}$ vertices, call the set W
S includes the shortest path trees from all vertices in W
for all vertices u:
    if there is an edge (u,w) s.t. w∈W,
        add (u,w) in S
    else include all edges (u,v) in S

#edges is $O(n \cdot n^{1/2})$

When the degree of u is $\Omega(n^{1/2})$, w.h.p. it is adjacent to a vertex in W



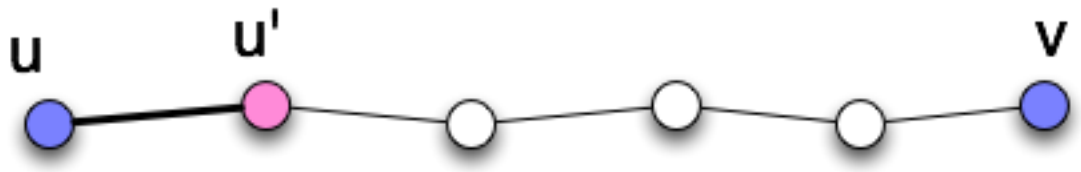
Randomly select $n^{1/2}$ vertices, call the set W
S includes the shortest path trees from all vertices in W
for all vertices u:
    if there is an edge (u,w) s.t. w∈W,
        add (u,w) in S
    else include all edges (u,v) in S

- Consider the shortest path from u to v in G:
  - If u or v is in W, the shortest path must also be in S, it is trivial



Randomly select $n^{1/2}$ vertices, call the set W
S includes the shortest path trees from all vertices in W
for all vertices u:
    if there is an edge (u,w) s.t. w∈W,
        add (u,w) in S
    else include all edges (u,v) in S

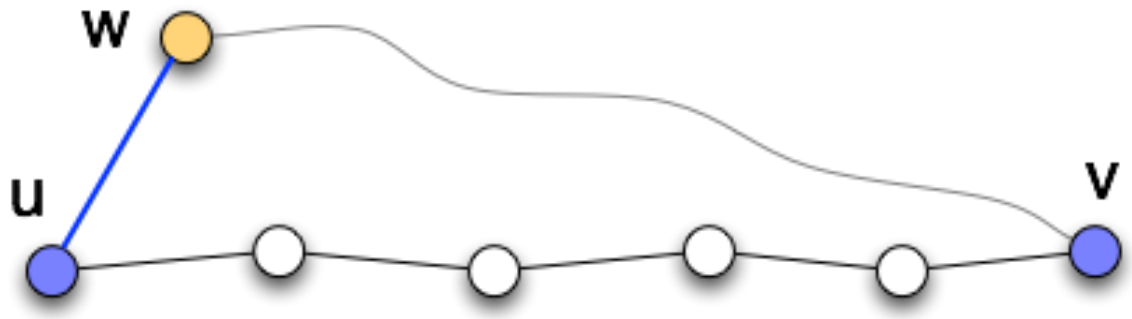
- Consider the shortest path from u to v in G:
  - If u or v is in W, the shortest path must also be in S, it is trivial
  - Otherwise if the first edge (u,u') ∈S, recursively consider the (u',v)





Randomly select $n^{1/2}$ vertices, call the set W
S includes the shortest path trees from all vertices in W
for all vertices u:
    if there is an edge (u,w) s.t. w∈W,
        add (u,w) in S
    else include all edges (u,v) in S

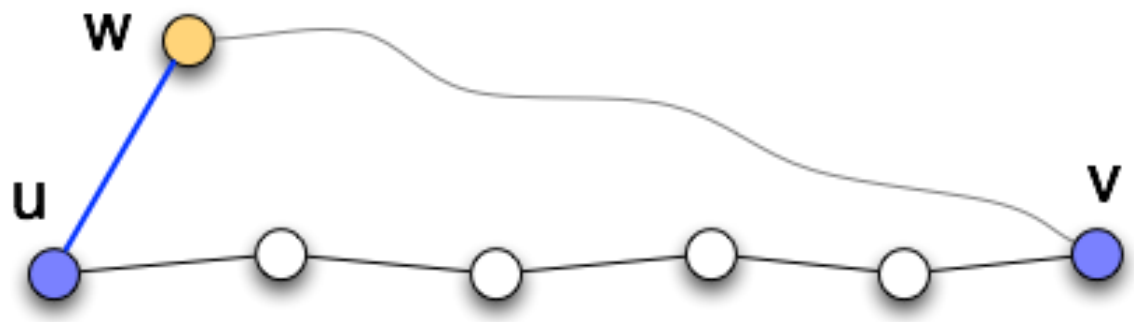
- if the first edge (u,u') ∉S, u is adjacent to a w∈W
- (u,w) and a shortest path from w to v is in S





Randomly select $n^{1/2}$ vertices, call the set W
S includes the shortest path trees from all vertices in W
for all vertices u:
    if there is an edge (u,w) s.t. w∈W,
        add (u,w) in S
    else include all edges (u,v) in S

- if the first edge (u,u') ∉S, u is adjacent to a w∈W
- (u,w) and a shortest path from w to v is in S



$d(w,v) \le d(w,u)+d(u,v)=d(u,v)+1$,
So $d_S(u,v) \le d(u,w)+d(w,v)=d(w,v)+1 \le d(u,v)+2$

2-additive spanner with $O(n^{3/2})$ edges

- The construction of the spanner S:

S=∅
Include every vertex in W with prob. $n^{-1/2}$
S includes the shortest path trees from all vertices in W
for all vertices u:
    for all edges (u,v) in G
        S includes (u,v)
        if v ∈ W, break

#edges associated with u:
Geometric distribution with $p=n^{-1/2}$
Expectation: $n^{1/2}$



Why randomness?

- We have proved a mathematical theorem:
  - Any undirected unweighted graph has a 2-additive spanner with $O(n^{3/2})$ edges
- By showing a randomized algorithm constructing it
  - Without randomness, we may need complicated mathematical proofs

We can construct it with high-probability ⇒ It must exist



Additive spanners

- 2-additive spanner with $O(n^{3/2})$ edges
  - 1999
- 4-additive spanner with $\tilde{O}(n^{7/5})$ edges
  - Chechik
  - Best student paper in SODA 2013
- 6-additive spanner with $O(n^{4/3})$ edges
  - Baswana, Kavitha, Mehlhorn, and Pettie
  - SODA 2005
- No further result for additive spanner with fewer edges!



Additive spanners

- 2-additive spanner with $O(n^{3/2})$ edges
  - 1999
- 4-additive spanner with $\tilde{O}(n^{7/5})$ edges
  - Chechik
  - Best student paper in SODA 2013
- 6-additive spanner with $O(n^{4/3})$ edges
  - Baswana, Kavitha, Mehlhorn, and Pettie
  - SODA 2005
- It is proved there is no constant additive spanner with $O(n^{4/3-\varepsilon})$ edges
  - "*The 4/3 Additive Spanner Exponent is Tight*" Abboud and Bodwin STOC 2016



Primality test

- Checking whether an integer N is a prime number
  - We want an algorithm polynomial of the length of the number, that is, $O(\log^c N)$
  - We will introduce a randomized polynomial algorithm

- Finding factors is not an efficient way!
  - no polynomial time algorithm for factoring known so far
  - but we can solve it if there are quantum computers



Fermat's little theorem

- Let $Z_p=\{0,\ldots,p-1\}$ and $Z^+_p=\{1,\ldots,p-1\}$ for any p
- Fermat's little theorem:

  If p is a prime, $\forall a \in Z^+_p$, $a^{p-1} \equiv 1 \pmod p$



Fermat's little theorem

- Let $Z_p=\{0,\ldots,p-1\}$ and $Z^+_p=\{1,\ldots,p-1\}$ for any p
- Fermat's little theorem:

  If p is a prime, $\forall a \in Z^+_p$, $a^{p-1} \equiv 1 \pmod p$

  - In fact, Z^+_p is a group under multiplication modulo p
  - so the order of any element is a factor of the order of the group



- Thus, maybe we can use Fermat's little theorem to test primality
- Randomly select $a \in Z^{+}_{p}$, and check whether $a^{p-1} \equiv 1 \pmod{p}$
  - computing a^{p-1} only takes O(log p) multiplications

- Problem: the converse of Fermat's little theorem is not always true!
  - there are composite numbers which can pass most Fermat test



Carmichael numbers

- In number theory, a **Carmichael number** is a composite number n which satisfies:

  $\forall a \in Z^{+}_{n}$ that are relatively prime to n, $a^{n-1} \equiv 1 \pmod{n}$

  - So it passes most Fermat tests
  - Also called "*absolute Fermat pseudoprimes*"
  - examples: $561 = 3 \times 11 \times 17$, $1105 = 5 \times 13 \times 17$, $1729 = 7 \times 13 \times 19$, $2465 = 5 \times 17 \times 29$ …

  - it's not rare: if C(X) denotes the number of Carmichael numbers ≤X

    $C(X) > X^{0.333367}$



Further property for prime numbers

- If p is odd, (p-1) is even, if $a^{p-1} \equiv 1 \pmod{p}$, we consider $a^{(p-1)/2} \pmod{p}$

- Theorem. Let p is an odd prime, for an even number k and any $a \in Z^+_p$, if $a^k \equiv 1 \pmod{p}$, then $a^{k/2} \equiv \pm 1 \pmod{p}$

Proof.

Let $b = a^{k/2} \bmod p$

since $b^2 = c \cdot p + 1$ for an integer c

$(b+1)(b-1) = c \cdot p$

So b can only be 1 or p-1



Algorithm for primality test

- Input: integer N
- Randomly select $a \in Z^+_N$
- Compute $a^{N-1} \bmod N$, if it's not 1, reject
- Let $k=N-1$
- while (k is even) do

  $k=k/2$

  If ($a^k \bmod N=-1$) accept

  If ($a^k \bmod N \neq 1$ or -1), reject
- accept.

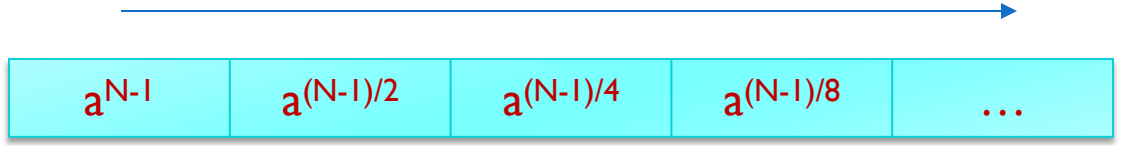
Time: polynomial on log N



Algorithm for primality test

Theorem. Let p is an odd prime, for an even number k and any $a \in Z^+_p$, if $a^k \equiv 1 \pmod p$, then $a^{k/2} \equiv \pm 1 \pmod p$

- Input: integer N
- Randomly select $a \in Z^+_N$
- Compute $a^{N-1} \bmod N$, if it's not 1, reject
- Let k=N-1
- while (k is even) do
  - k=k/2
  - If ($a^k \bmod N$=-1) accept
  - If ($a^k \bmod N \neq 1$ or -1), reject
- accept.



Find the first one which is not 1 mod p, if it is also not -1 mod p, then reject



Properties

1. If N is an odd prime, the algorithm always accepts
   - By the theorem we've already seen
2. If N is an odd composite, there must be an $a \in Z^{+}_{N}$ which is relatively prime to N s.t. the algorithm rejects

3. From (2), we can see at least one half of elements Z^{+}_{N} can make the algorithm reject
   - thus, we have a randomized algorithm



- 2. If N is an odd composite, there must be an $a \in Z^{+}_{N}$ which is relatively prime to N s.t. the algorithm rejects

  - $N=p^{i}$ for some prime p and $i \geq 2$, or $N=q \cdot r$ for relatively prime q and r
  - If $N=p^{i}$, consider $t=1+p^{i-1}$, then $t^{N} \equiv 1 \pmod{N}$, so $t^{N-1} \not\equiv 1 \pmod{N}$,



- 2. If N is an odd composite, there must be an $a \in Z^+_N$ which is relatively prime to N s.t. the algorithm rejects

- $N=p^i$ for some prime p and $i \geq 2$, or $N=q \cdot r$ for relatively prime q and r
- If $N=p^i$, consider $t=1+p^{i-1}$, then $t^N \equiv 1 \pmod N$, so $t^{N-1} \not\equiv 1 \pmod N$,
- If $N=q \cdot r$, find an $h^k \equiv -1 \pmod N$ with largest $k=(N-1)/2^i$,

Why must h exist?

by the Chinese remainder theorem, these exists a t such that:

$$t \equiv h \pmod q$$
$$t \equiv 1 \pmod r$$

therefore,

$$t^k \equiv -1 \pmod q$$
$$t^k \equiv 1 \pmod r$$

so $t^{2k} \equiv 1 \pmod N$ but $t^k \not\equiv \pm 1 \pmod N$

$t^{N-1} \equiv 1 \pmod N$ so t is relatively prime to N



- Randomly select $a \in Z^+_N$
- Compute $a^{N-1} \bmod N$, if it's not 1, reject
- Let $k=N-1$
- while (k is even) do
  - $k=k/2$
  - If ($a^k \bmod N=-1$) accept
  - If ($a^k \bmod N \neq 1$ or -1), reject
- accept.

- 3. From (2), we can see at least one half of elements Z^+_N can make the algorithm reject

- For $t \in Z^+_N$ that are not relatively prime to N, of course reject, so we only consider numbers that are relatively prime to N
- For such a $t \in Z^+_N$ we get in (2)
  - if we have two $a,b \in Z^+_N$ which make the algorithm accept
  - then $a \cdot t$ and $b \cdot t$ are distinct elements that make the algorithm reject!
- Thus, #accept elements ≤ #reject elements



find an $h^k \equiv -1 \pmod N$ with largest $k=(N-1)/2^i$

- Randomly select $a \in Z^+_N$
- Compute $a^{N-1} \bmod N$, if it's not 1, reject
- Let $k=N-1$
- while (k is even) do
  - $k=k/2$
  - If ($a^k \bmod N=-1$) accept
  - If ($a^k \bmod N \neq 1$ or -1), reject
- accept.

- 3. From (2), we can see at least one half of elements Z^+_N can make the algorithm reject

We only consider numbers that are relative prime to N, and $N=q \cdot r$
For such a $t \in Z^+_N$ we get in (2) (with largest k)
if we have two $a,b \in Z^+_N$ which makes the algorithm accept
so $a^k \equiv \pm 1 \pmod N$, $a^{2k} \equiv 1 \pmod N$, and the same as b
then $a \cdot t$ and $b \cdot t$ are distinct elements that makes the algorithm reject!
$(a \cdot t)^k \not\equiv \pm 1 \pmod N$, $(a \cdot t)^{2k} \equiv 1 \pmod N$, the same as $b \cdot t$
if $a \cdot t \equiv b \cdot t \pmod N$, $(a-b)t=c \cdot N$ for some integer c, which is impossible
Thus, #accept elements ≤ #reject elements



Primality test

- Randomly select $a \in Z^+_N$
- Compute $a^{N-1} \bmod N$, if it's not 1, reject
- Let $k=N-1$
- while (k is even) do
  - $k=k/2$
  - If ($a^k \bmod N=-1$) accept
  - If ($a^k \bmod N \neq 1$ or -1), reject
- accept.

- When N is an odd prime, always accept
- When N is an odd composite, reject with prob. $\geq 1/2$



Probabilistic TM Facts

- Each "branch" in the TM's computation has a probability
- Hence, on a given input it:
  - May have different running times
  - May not halt
- Therefore, it may accept the input in a given execution, but reject in another execution
- Time and space complexity can be measured using the worst case computation branch



TM Specifics

- There are (at least) three tapes
  - 1st Tape holds the input
  - 2nd Tape (also known as the random tape) is covered randomly (and independently) with 0's and 1's
    - ½ probability of a 0
    - ½ probability of a 1
  - 3rd Tape is used as the working tape



Randomized complexity classes

- model: probabilistic Turing Machine
  - deterministic TM with additional read-only tape containing "coin flips"
- **BPP** (Bounded-error Probabilistic Poly-time)
  - L ∈ **BPP** if there is a p.p.t. TM M:

    $x \in L \Rightarrow Pr_y[M(x,y) \text{ accepts}] \geq 2/3$

    $x \notin L \Rightarrow Pr_y[M(x,y) \text{ rejects}] \geq 2/3$
  - "p.p.t" = probabilistic polynomial time



A Problem in BPP:

- Can be solved by an algorithm that is allowed to make random decisions (called coin-flips)
- Guaranteed to run in polynomial time
- On a given run of the algorithm, it has (at most) 1/3 probability of giving an incorrect answer
- These algorithms are known as probabilistic algorithms



Why 1/3?

- Actually, this is arbitrary
- In fact can be any constant between [0,½) (as long as it is independent of the input)
- error reduction



Error reduction for BPP

- given L, and p.p.t. TM M:

$$x \in L \Rightarrow Pr_y[M(x,y) \text{ accepts}] \geq \tfrac{1}{2} + \varepsilon$$

$$x \notin L \Rightarrow Pr_y[M(x,y) \text{ rejects}] \geq \tfrac{1}{2} + \varepsilon$$

- new p.p.t. TM M':
  - simulate M k/ε^2 times, each time with independent coin flips
  - accept if **majority** of simulations accept
  - otherwise reject



Error reduction for BPP

- X_i random variable indicating "correct" outcome in i-th simulation (out of $m = k/\varepsilon^2$)
  - $\Pr[X_i = 1] \geq \frac{1}{2} + \varepsilon$
  - $\Pr[X_i = 0] \leq \frac{1}{2} - \varepsilon$
- $E[X_i] \geq \frac{1}{2}+\varepsilon$
- $X = \Sigma_i X_i$
- $\mu = E[X] \geq (\frac{1}{2} + \varepsilon)m$

- Chernoff bound: $\Pr[X \leq m/2] \leq 2^{-\Omega(\varepsilon^2 \mu)}$



Error reduction for BPP

- X_i random variable indicating "correct" outcome in i-th simulation (out of $m = k/\varepsilon^2$)
  - $\Pr[X_i = 1] \geq \frac{1}{2} + \varepsilon$
  - $\Pr[X_i = 0] \leq \frac{1}{2} - \varepsilon$

- Here we calculate it directly:
- It's a binomial distribution consisting of m experiments
- Let Y denote the number of wrong outcomes
- $p \leq \frac{1}{2} - \varepsilon$ is the probability of wrong outcome each time



Error reduction for BPP

- X_i random variable indicating "correct" outcome in i-th simulation (out of $m = k/\varepsilon^2$)
  - $\Pr[X_i = 1] \geq \frac{1}{2} + \varepsilon$
  - $\Pr[X_i = 0] \leq \frac{1}{2} - \varepsilon$

- Here we calculate it directly:
- It's a binomial distribution consisting of m experiments
- Let Y denote the number of wrong outcomes
- $p \leq \frac{1}{2} - \varepsilon$ is the probability of wrong outcome each time

$$\Pr[Y = k] = \binom{m}{k} p^k (1-p)^{m-k}$$



Error reduction for BPP

- X_i random variable indicating "correct" outcome in i-th simulation (out of $m = k/\varepsilon^2$)
  - $\Pr[X_i = 1] \geq \frac{1}{2} + \varepsilon$
  - $\Pr[X_i = 0] \leq \frac{1}{2} - \varepsilon$

- Here we calculate it directly:
- It's a binomial distribution consisting of m experiments
- Let Y denote the number of wrong outcomes
- $p \leq \frac{1}{2} - \varepsilon$ is the probability of wrong outcome each time

$$\Pr[Y = k] = \binom{m}{k} p^k (1-p)^{m-k}$$

$$\Pr[Y \geq m/2] \leq 2^m p^{m/2} (1-p)^{m/2} \leq \left(4p(1-p)\right)^{m/2} \leq \left(1-4\varepsilon^2\right)^{m/2} = \left(1-4\varepsilon^2\right)^{\frac{k}{2\varepsilon^2}} \approx e^{-2k}$$



Error reduction for BPP

$$x \in L \Rightarrow Pr_y[M(x,y) \text{ accepts}] \geq \tfrac{1}{2} + \varepsilon$$
$$x \notin L \Rightarrow Pr_y[M(x,y) \text{ rejects}] \geq \tfrac{1}{2} + \varepsilon$$

- if $x \in L$

$$Pr_{y'}[M'(x, y') \text{ accepts}] \geq 1 - e^{-\Omega(k)}$$

- if $x \notin L$

$$Pr_{y'}[M'(x,y') \text{ rejects}] \geq 1 - e^{-\Omega(k)}$$

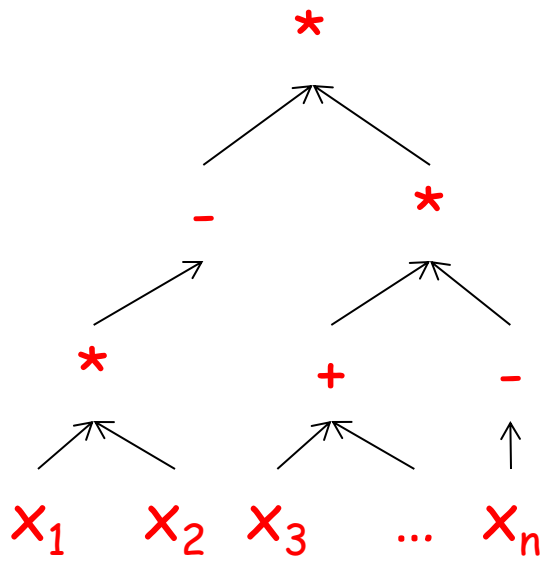
- with only k/ε^2 repetitions



An example: polynomial identity testing

- Given: polynomial $p(x_1, x_2, \ldots, x_n)$ as arithmetic formula (fan-out 1):

  - multiplication (fan-in 2)
  - addition (fan-in 2)
  - negation (fan-in 1)





Polynomial identity testing

- Question: Is p identically zero?
  - i.e., is $p(\mathbf{x}) = 0$ for all $\mathbf{x} \in \mathbf{R}^n$

- "polynomial identity testing" because given two polynomials p, q, we can check the identity $p \equiv q$ by checking if $p - q \equiv 0$



Possible solutions

- Open all parentheses
  - e.g. $(x_1+x_2)(x_1+x_2-3x_3)(x_1+5x_3-x_4)+(x_2+x_3)(x_1-x_3)(x_1+5x_3-x_4)+\ldots$
  - $\prod_i(1+x_i)$ will have exponential number of terms

- can randomness help?
  - i.e., flip coins, allow small probability of wrong answer



Polynomial identity testing

Lemma (Schwartz-Zippel): Let

$$p(x_1, x_2, \ldots, x_n)$$

be a total degree d non-zero polynomial, and let S be any subset of **integers** Then,

$$Pr_{r_1,r_2,\ldots,r_n \in S}[\, p(r_1, r_2, \ldots, r_n) = 0] \leq d/|S|$$



Polynomial identity testing

- Proof:
  - induction on number of variables n
  - base case: n = 1, p is univariate polynomial of degree at most d
  - By the fundamental theorem of algebra, it has at most d roots, so

$$\Pr[\, p(r_1) = 0] \leq d/|S|$$



Polynomial identity testing

- write $p(x_1, x_2, \ldots, x_n)$ as

$$p(x_1, x_2, \ldots, x_n) = \Sigma_i (x_1)^i p_i(x_2, \ldots, x_n)$$

- k = max. i for which $p_i(x_2, \ldots, x_n)$ not identically zero
- by induction hypothesis:

$$\Pr[\, p_k(r_2, \ldots, r_n) = 0] \leq (d\text{-}k)/|S|$$

- whenever $p_k(r_2, \ldots, r_n) \neq 0$, $p(x_1, r_2, \ldots, r_n)$ is a univariate polynomial of degree k

$$\Pr[p(r_1, r_2, \ldots, r_n)=0 \mid p_k(r_2, \ldots, r_n) \neq 0] \leq k/|S|$$



Polynomial ident

Note: we can add these probabilities because

$$\Pr[E_1] = \Pr[E_1|E_2]\Pr[E_2] + \Pr[E_1|\sim E_2]\Pr[\sim E_2]$$
$$\leq \Pr[E_2] + \Pr[E_1|\sim E_2]$$

- write $p(x_1, x_2, \ldots, x_n)$ as

$$p(x_1, x_2, \ldots, x_n) = \Sigma_i\, (x_1)^i\, p_i(x_2, \ldots, x_n)$$

- k = max. i for which $p_i(x_2, \ldots, x_n)$ not identically zero
- by induction hypothesis:

$$\Pr[\, p_k(r_2, \ldots, r_n) = 0] \leq (d\text{-}k)/|S|$$

- whenever $p_k(r_2, \ldots, r_n) \neq 0$, $p(x_1, r_2, \ldots, r_n)$ is a univariate polynomial of degree k

$$\Pr[p(r_1, r_2, \ldots, r_n)=0 \mid p_k(r_2, \ldots, r_n) \neq 0] \leq k/|S|$$

- conclude:

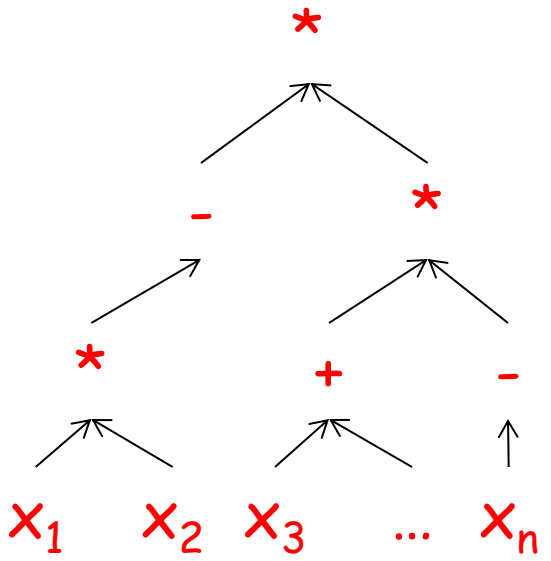
$$\Pr[\, p(r_1, \ldots, r_n) = 0] \leq (d\text{-}k)/|S| + k/|S| = d/|S|$$



Polynomial identity testing

- Given: polynomial $p(x_1, x_2, \ldots, x_n)$ as a formula

- Is p identically zero?



- Note: degree d is at most the size of input, since it is a formula



Polynomial identity testing

- randomized algorithm: field **F,** pick a subset S ⊂ **F** of size 3d
  - pick $r_1, r_2, \ldots, r_n$ from S uniformly at random
  - if $p(r_1, r_2, \ldots, r_n) = 0$, answer "yes"
  - if $p(r_1, r_2, \ldots, r_n) \neq 0$, answer "no"

- if p identically zero, never wrong
- if not, Schwartz-Zippel ensures probability of error at most 1/3

- Note: computing r^d only takes O(log d) multiplications



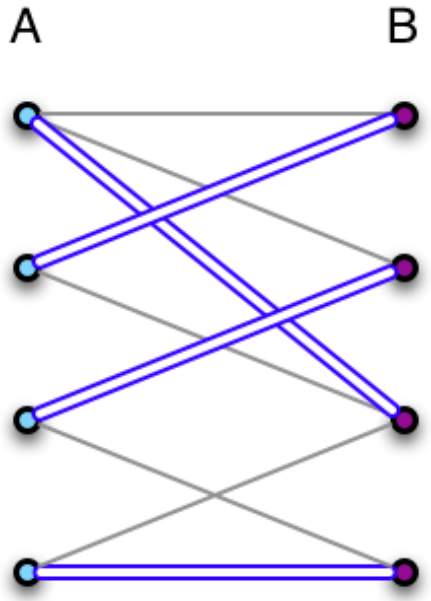
Another example

- Perfect matching in bipartite graph:
  - Whether there is a perfect matching in G

- By determinant of adjacency matrix

$$\det(A) = \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma) \prod_{i=1}^{n} a_{i,\sigma_i}$$

  - σ is the permutation of {1,…,n}





Another example

- Perfect matching in bipartite graph:
  - Whether there is a perfect matching in G

- By determinant of adjacency matrix

$$\det(A) = \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma) \prod_{i=1}^{n} a_{i,\sigma_i}$$

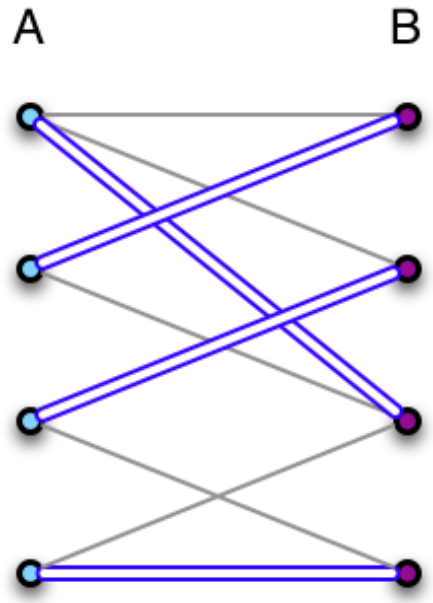
  - σ is the permutation of {1,…,n}
  - sgn(σ) is +1 or -1

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix}$$

Permutation means every row and column contains exactly one element



- A perfect matching in a bipartite graph with n vertices each side can also be seen as a permutation of {1,…,n}

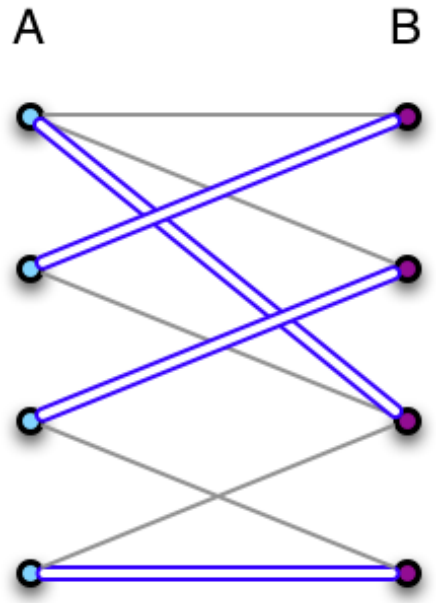




- A perfect matching in a bipartite graph with n vertices each side can also be seen as a permutation of {1,…,n}

- Construct an n×n matrix X:

$$\begin{bmatrix} x_{11} & x_{12} & x_{13} & 0 \\ x_{21} & 0 & x_{23} & 0 \\ 0 & x_{32} & 0 & x_{34} \\ 0 & 0 & x_{43} & x_{44} \end{bmatrix}$$



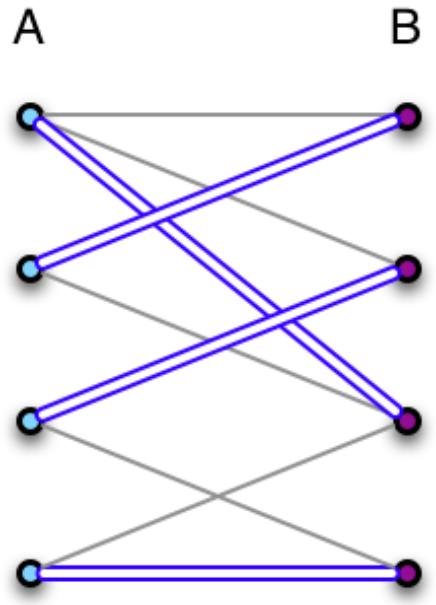
  - So $x_{ij}=0$ if there is no edge from A_i to B_j



- A perfect matching in a bipartite graph with n vertices each side can also be seen as a permutation of {1,…,n}

- Construct an n×n matrix X:

$$\begin{bmatrix} x_{11} & x_{12} & x_{13} & 0 \\ x_{21} & 0 & x_{23} & 0 \\ 0 & x_{32} & 0 & x_{34} \\ 0 & 0 & x_{43} & x_{44} \end{bmatrix}$$



A perfect matching corresponds to a non-zero term in the determinant



- A perfect matching in a bipartite graph with n vertices each side can also be seen as a permutation of {1,…,n}

- Construct an n×n matrix X:

$$\begin{bmatrix} x_{11} & x_{12} & x_{13} & 0 \\ x_{21} & 0 & x_{23} & 0 \\ 0 & x_{32} & 0 & x_{34} \\ 0 & 0 & x_{43} & x_{44} \end{bmatrix}$$

$$\det(X) = \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma) \prod_{i=1}^{n} x_{i,\sigma_i}$$

  - From the definition of determinant, det(X) ≡ 0 iff there is no perfect matching



- The determinant is a polynomial of degree n

- As in the polynomial identity testing, we can randomly pick each nonzero x_{ij} from {1,…,3n}, and compute det(X)
  - Computing det(X) takes $O(n^{\omega})$ time, the same as matrix multiplication



Observation

- This example is of course in BPP

$$x \in L \Rightarrow Pr_y[M(x,y) \text{ accepts}] \geq 2/3$$

$$x \notin L \Rightarrow Pr_y[M(x,y) \text{ rejects}] \geq 2/3$$



Observation

- This example is of course in BPP

$$x \in L \Rightarrow Pr_y[M(x,y) \text{ accepts}] \geq 2/3$$

$$x \notin L \Rightarrow Pr_y[M(x,y) \text{ rejects}] = 1$$

  - When there is no perfect matching, the determinant is identically zero, so the algorithm always return 0



RP

- Randomized Polynomial time
- A class of problems that will run in polynomial time on a probabilistic TM with the following properties: if the correct answer is
  - no, always return no
  - yes, return yes with probability at least ½



Randomized complexity classes

- **RP** (Random Polynomial-time)
  - L ∈ **RP** if there is a p.p.t. TM M:

    $x \in L \Rightarrow Pr_y[M(x,y) \text{ accepts}] \geq ½$

    $x \notin L \Rightarrow Pr_y[M(x,y) \text{ rejects}] = 1$
- **coRP** (complement of Random Polynomial-time)
  - L ∈ **coRP** if there is a p.p.t. TM M:

    $x \in L \Rightarrow Pr_y[M(x,y) \text{ accepts}] = 1$

    $x \notin L \Rightarrow Pr_y[M(x,y) \text{ rejects}] \geq ½$



Randomized complexity classes

One more important class:

- **ZPP** (Zero-error Probabilistic Poly-time)
  - L ∈ **ZPP** if there is a probabilistic TM M:

$$x \in L \Rightarrow Pr_y[M(x,y) \text{ accepts}] = 1$$

$$x \notin L \Rightarrow Pr_y[M(x,y) \text{ rejects}] = 1$$

The expected running time of M is polynomial!



Another definition

- **ZPP = RP ∩ coRP**



Relationship Between RP and ZPP

- ZPP = RP ∩ co-RP
- Proof Part 1: RP ∩ co-RP ⊆ ZPP
  - Let L be a language recognized by RP algorithm A and co-RP algorithm B

L ∈ **RP** if there is a p.p.t. TM M:
$x \in L \Rightarrow Pr_y[M(x,y) \text{ accepts}] \geq ½$
$x \notin L \Rightarrow Pr_y[M(x,y) \text{ rejects}] = 1$
L ∈ **coRP** if there is a p.p.t. TM M:
$x \in L \Rightarrow Pr_y[M(x,y) \text{ accepts}] = 1$
$x \notin L \Rightarrow Pr_y[M(x,y) \text{ rejects}] \geq ½$



Relationship Between RP and ZPP

- ZPP = RP ∩ co-RP
- Proof Part 1: RP ∩ co-RP ⊆ ZPP
  - Let L be a language recognized by RP algorithm A and co-RP algorithm B

Run w on A and B simultaneously
If A accepts, the answer must be YES
If B rejects, the answer must be NO
Otherwise, repeat

  - In each iteration the probability of halting is ≥50%
  - So the chance of having the k-th repetition shrinks exponentially. Therefore, the expected running time is polynomial



Relationship Between RP and ZPP

- ZPP = RP ∩ co-RP
- Proof Part 2: ZPP ⊆ RP ∩ co-RP



Relationship Between RP and ZPP

- ZPP = RP ∩ co-RP
- Proof Part 2: ZPP ⊆ RP ∩ co-RP
  - Let C be an algorithm in ZPP
  - Construct the RP algorithm using C:
    - Run C for (at least) double its expected running time
    - If it gives an answer, that must be the answer
    - If it doesn't given an answer before the algorithm stops, then the answer is ?



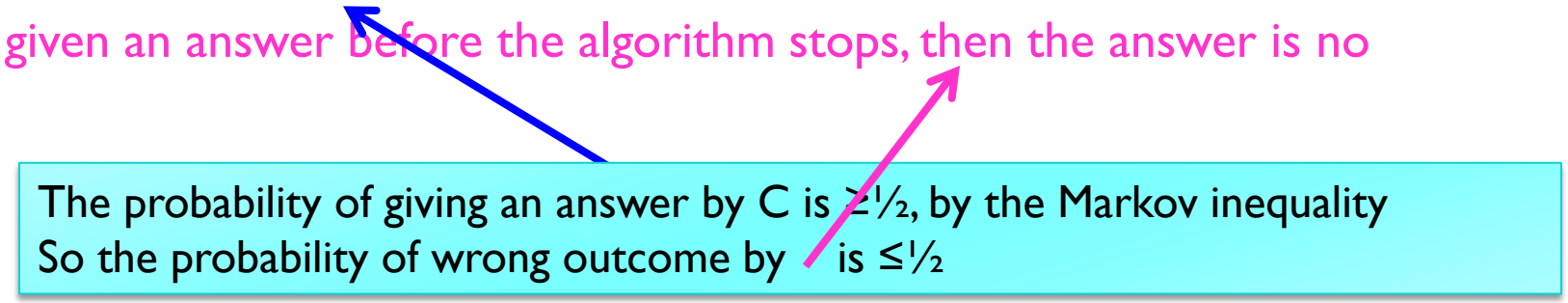
Relationship Between RP and ZPP

- ZPP = RP ∩ co-RP
- Proof Part 2: ZPP ⊆ RP ∩ co-RP
  - Let C be an algorithm in ZPP
  - Construct the RP algorithm using C:
    - Run C for (at least) double its expected running time
    - If it gives an answer, that must be the answer
    - If it doesn't given an answer before the algorithm stops, then the answer is no



Relationship Between RP and ZPP

- ZPP = RP ∩ co-RP
- Proof Part 2: ZPP ⊆ RP ∩ co-RP
  - Let C be an algorithm in ZPP
  - Construct the RP algorithm using C:
    - Run C for (at least) double its expected running time
    - If it gives an answer, that must be the answer
    - If it doesn't given an answer before the algorithm stops, then the answer is no



  - The co-RP algorithm can be constructed similarly
  - Therefore, we can conclude that ZPP ⊆ RP ∩ co-RP



Monte Carlo vs. Las Vegas algorithms

Monte Carlo

- a randomized algorithm whose running time is deterministic, but whose output may be incorrect with a certain (typically small) probability

Las Vegas

- a randomized algorithm that always gives correct results; it gambles only with the resources used for the computation
  - e.g. Quicksort



Monte Carlo vs. Las Vegas algorithms

Monte Carlo

- a randomized algorithm whose running time is deterministic, but whose output may be incorrect with a certain (typically small) probability
- RP, co-RP, BPP, …

Las Vegas

- a randomized algorithm that always gives correct results; it gambles only with the resources used for the computation
- ZPP
  - e.g. Quicksort



Both names are referred to famous gambling sites

Monte Carlo

- in the microstate of Monaco
- area of Monaco: 2.02km^2



Las Vegas

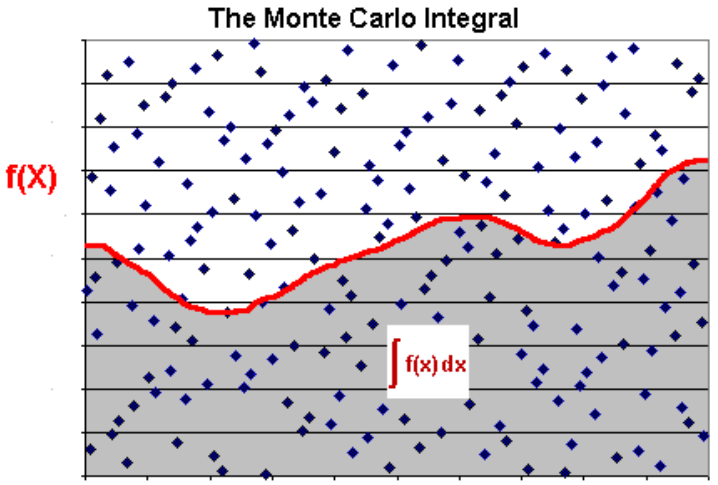
- In Nevada desert
- most famous gambling city



Monte Carlo method

- Usually denotes *stochastic simulation*
- a broad class of computational algorithms that rely on repeated random sampling to obtain numerical results

- e.g. Monte Carlo integration





What We Can Also Conclude

- Of course ZPP=coZPP

- As seen in the proof of ZPP = RP ∩ co-RP we can conclude that
  - ZPP ⊆ RP
  - ZPP ⊆ co-RP



What We Can Also Conclude

- Of course ZPP=coZPP

- As seen in the proof of ZPP = RP ∩ co-RP we can conclude that
  - ZPP ⊆ RP
  - ZPP ⊆ co-RP

- Also trivially RP, co-RP ⊆BPP



Relationship Between P and ZPP

- $P \subseteq ZPP$
- Proof
  - trivial



Relationship Between NP and RP

- RP ⊆ NP
- Proof

$L \in RP$:
$x \in L \Rightarrow Pr_y[M(x,y) \text{ accepts}] \geq ½$
$x \notin L \Rightarrow Pr_y[M(x,y) \text{ rejects}] = 1$

  - So $x \in L \Leftrightarrow \exists y[M(x,y) \text{ accepts}]$



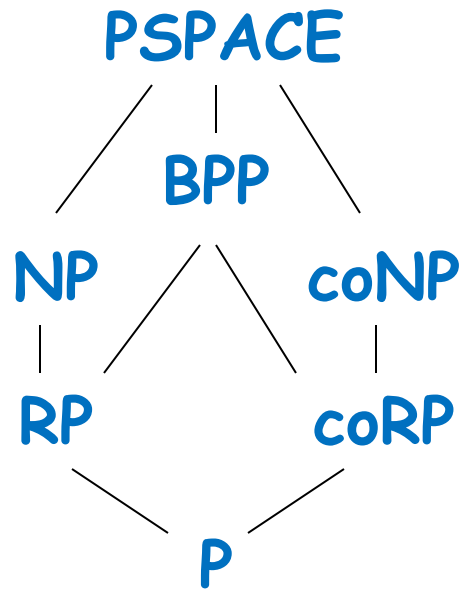
Relationship to other classes

- ZPP, RP, coRP, BPP, contain **P**
  - they can simply ignore the tape with coin flips
- all are in **PSPACE**
  - can exhaustively try all strings y
  - count accepts/rejects; compute probability
- **RP** $\subseteq$ **NP** (and **coRP** $\subseteq$ **coNP**)
  - multitude of accepting computations
  - **NP** requires only one



Where Does BPP Fit in?

- It is still an open question whether NP is a subset of BPP or BPP is a subset of NP
- However, RP is a subset of BPP





BPP

- Since we don't know the relationship of BPP and NP
- What about the relationship of BPP and polynomial hierarchy PH?

- Review: L in $\sum_2^p$ iff

$$x \in L \Leftrightarrow \exists u_1 \in \{0,1\}^{q(|x|)} \forall u_2 \in \{0,1\}^{q(|x|)} M(x, u_1, u_2) = 1$$



The polynomial hierarchy PH

- For i≥1, a language L is in $\sum_i^p$ if there exists a polynomial time TM M and a polynomial q s.t.

$$x \in L \Leftrightarrow \exists u_1 \in \{0,1\}^{q(|x|)} \forall u_2 \in \{0,1\}^{q(|x|)} \ldots Q_i u_i \in \{0,1\}^{q(|x|)} M(x, u_1, \ldots, u_i) = 1$$

  - where Q_i is ∃ when i is odd; is ∀ when i is even.

- Define $PH = \cup_i \sum_i^p$
- Define $\prod_i^p = co \sum_i^p$, which is:

$$x \in L \Leftrightarrow \forall u_1 \in \{0,1\}^{q(|x|)} \exists u_2 \in \{0,1\}^{q(|x|)} \ldots Q_i u_i \in \{0,1\}^{q(|x|)} M(x, u_1, \ldots, u_i) = 1$$

- **Note that $\sum_i^p \subseteq \prod_{i+1}^p \subseteq \sum_{i+2}^p$, so $PH = \cup_i \prod_i^p$**



An example

- An independent set is a subset of vertices so that no pair is connected
- Exact independent set problem: given <G,k>, whether the largest independent set in G has size exactly k
  - in $\sum_2^p$ or $\prod_2^p$?



An example

- An independent set is a subset of vertices so that no pair is connected
- Exact independent set problem: given <G,k>, whether the largest independent set in G has size exactly k
  - in both $\sum_2^p$ and $\prod_2^p$

$$\exists S_1 \forall S_2 \, [|S_1|=k \wedge S_1 \text{ is independent} \wedge (|S_2|>k \rightarrow S_2 \text{ is not independent})]$$



Collapse of hierarchy

- If P=NP, then PH=P, that is, the hierarchy collapses to P



Collapse of hierarchy

- If P=NP, then PH=P, that is, the hierarchy collapses to P
- Proof:
  - $\sum_1^P = NP = P$
  - Proof by induction: if $\sum_{i-1}^P = P$, then $\sum_i^P = P$



Collapse of hierarchy

- If P=NP, then PH=P, that is, the hierarchy collapses to P
- Proof:
  - $\sum_1^P$ =NP=P
  - If $\sum_{i-1}^P$ =P, then $\sum_i^P$ =P
  - Definition of $\sum_i^P$:

$$x \in L \Leftrightarrow \exists u_1 \in \{0,1\}^{q(|x|)} \forall u_2 \in \{0,1\}^{q(|x|)} \ldots Q_i u_i \in \{0,1\}^{q(|x|)} M(x, u_1, \ldots, u_i) = 1$$

  - Define L':

$$(x, u_1) \in L' \Leftrightarrow \forall u_2 \in \{0,1\}^{q(|x|)} \ldots Q_i u_i \in \{0,1\}^{q(|x|)} M(x, u_1, \ldots, u_i) = 1$$

  - then L' $\in \prod_{i-1}^P$ =co $\sum_{i-1}^P$ =P
  - so $x \in L \Leftrightarrow \exists u_1 \in \{0,1\}^{q(|x|)} [(x, u_1) \in L']$, L∈NP=P



Collapse of hierarchy

- If P=NP, then PH=P, that is, the hierarchy collapses to P

- Similarly, if $\sum_i^p = \prod_i^p$, $PH = \sum_i^p$, the hierarchy collapses to $\sum_i^p$



BPP ⊆ PH

- we even don't know whether **BPP** different from **EXP**

Theorem*[1983]*: **BPP**⊆ ($\Pi_2 \cap \Sigma_2$)

(Sipser–Gács–Lautemann theorem)

- don't know $\Pi_2 \cap \Sigma_2$ different from **EXP** but believe much weaker



BPP ⊆ PH

- Proof:
  - by error reduction,
  - **BPP** language L: p.p.t. TM M:

$$x \in L \Rightarrow \Pr_y[M(x,y) \text{ accepts}] \geq 1 - \frac{1}{2m}$$

$$x \notin L \Rightarrow \Pr_y[M(x,y) \text{ accepts}] \leq \frac{1}{2m}$$

    - use m random bits (|y| = m)

$\Pr_y[M(x,y) \text{ incorrect}] \leq ½ + \varepsilon$
$\Pr_{y'}[M'(x, y') \text{ correct}] \geq 1 - e^{-\Omega(k)}$
with only k/ε^2 repetitions
We can make m large enough to get this



BPP ⊆ PH

- **BPP** language L: p.p.t. TM M:

$$x \in L \Rightarrow Pr_y[M(x,y) \text{ accepts}] \geq 1 - \frac{1}{2m}$$

$$x \notin L \Rightarrow Pr_y[M(x,y) \text{ accepts}] \leq \frac{1}{2m}$$

  - use m random bits (|y| = m)

- Claim:

$$x \in L \Leftrightarrow (\exists y_1,\ldots,y_m)(\forall z)[\text{one of } M(x,y_i \oplus z) \text{ accepts}]$$

  - here $y_1, \ldots, y_m, z$ are strings in $\{0,1\}^m$



$x \in L \Rightarrow Pr_y[M(x,y) \text{ accepts}] \geq 1 - \frac{1}{2m}$

$x \notin L \Rightarrow Pr_y[M(x,y) \text{ accepts}] \leq \frac{1}{2m}$

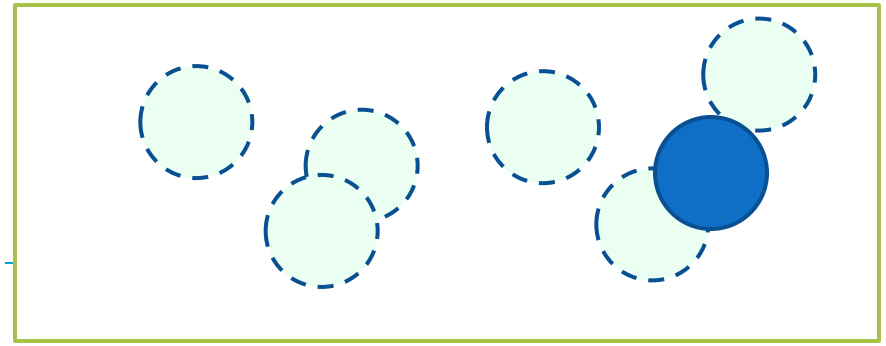
- use m random bits (|y| = m)

- Claim:

$x \in L \Rightarrow (\exists y_1,\ldots,y_m)(\forall z)[\text{one of } M(x,y_i \oplus z) \text{ accepts}]$

$x \notin L \Rightarrow (\forall y_1,\ldots,y_m)(\exists z)[\text{none of } M(x,y_i \oplus z) \text{ accepts}]$

- If $x \notin L$, only $\leq \frac{1}{2m} 2^m$ of random strings y can make M(x,y) accept
- So for any y_i, the number of z's which make $M(x,y_i \oplus z)$ accept is $\leq \frac{1}{2m} 2^m$
- for all $y_1,\ldots,y_m$, the number of z's which make one of $M(x,y_i \oplus z)$ accept is $\leq \frac{1}{2} 2^m$
- so for any $y_1,\ldots,y_m$, there exists z such that all of $M(x,y_i \oplus z)$ reject



$x \in L \Rightarrow Pr_y[M(x,y) \text{ accepts}] \geq 1 - \frac{1}{2m}$

$x \notin L \Rightarrow Pr_y[M(x,y) \text{ accepts}] \leq \frac{1}{2m}$

- use m random bits (|y| = m)

- Claim:

$$x \in L \Rightarrow (\exists y_1,\ldots,y_m)(\forall z)[\text{one of } M(x,y_i \oplus z) \text{ accepts}]$$

- If $x \in L$, for any z:
  - $Pr_{yi}[M(x,y_i \oplus z) \text{ rejects}] \leq \frac{1}{2m}$
  - $Pr_{y1,y2,\ldots,ym}[\text{all of } M(x,y_i \oplus z) \text{ rejects}] \leq \left(\frac{1}{2m}\right)^m$



$x \in L \Rightarrow Pr_y[M(x,y) \text{ accepts}] \geq 1 - \frac{1}{2m}$

$x \notin L \Rightarrow Pr_y[M(x,y) \text{ accepts}] \leq \frac{1}{2m}$

- use m random bits (|y| = m)

- Claim:

$x \in L \Rightarrow (\exists y_1,\ldots,y_m)(\forall z)[\text{one of } M(x,y_i \oplus z) \text{ accepts}]$

- If $x \in L$, for any z:
  - $Pr_{yi}[M(x,y_i \oplus z) \text{ rejects}] \leq \frac{1}{2m}$
  - $Pr_{y1,y2,\ldots,ym}[\text{all of } M(x,y_i \oplus z) \text{ rejects}] \leq \left(\frac{1}{2m}\right)^m$
- since there are 2^m such z's
  - $Pr_{y1,y2,\ldots,ym}[\exists z \text{ s.t. all of } M(x,y_i \oplus z) \text{ rejects}] \leq 2^m \cdot \left(\frac{1}{2m}\right)^m < 1$
  - $Pr_{y1,y2,\ldots,ym}[\forall z \text{ s.t. one of } M(x,y_i \oplus z) \text{ accepts}] > 0$
- so we get our claim



BPP ⊆ PH

- given **BPP** language L: p.p.t. TM M:

$$x \in L \Rightarrow Pr_y[M(x,y) \text{ accepts}] \geq 1 - \frac{1}{2m}$$

$$x \notin L \Rightarrow Pr_y[M(x,y) \text{ accepts}] \leq \frac{1}{2m}$$

  - use m random bits (|y| = m)

showed: $x \in L \Leftrightarrow (\exists y_1,\ldots,y_m)(\forall z)[\text{one of } M(x,y_i \oplus z) \text{ accepts}]$

- thus **BPP** ⊆ Σ_2
- **BPP** closed under complement ⇒ **BPP** ⊆ Π_2
- conclude: **BPP** ⊆ ($\Pi_2 \cap \Sigma_2$)

- So if P=NP, PH collapses to P, then P=BPP



- Two important problems by randomized algorithms are shown to have deterministic algorithms:
- Primality:
  - BPP: Solovay–Strassen primality test [1977]
  - P: AKS primality test [2002]
- UPATH: undirected connectivity
  - RL: by random walks [1979]
  - L: by expander, Reingold [2005]
- OPEN: whether BPP=P



PRIME is in P

AKS primality test

- The first deterministic polynomial time algorithm *(proven to be correct)* for primality test was given by Agrawal, Kayal and Saxena in 2002
  - They received Gödel prize in 2006



Basic ideas

- Consider polynomial of X
- THM. If a is relatively prime to n, then n is prime if and only if

$$(X-a)^n \equiv X^n - a \pmod{n}$$

- Proof.

$(X-a)^n = \sum_{i=0}^{n}\binom{n}{i}X^i(-a)^{n-i} = X^n + (-a)^n + \sum_{i=1}^{n-1}\binom{n}{i}X^i(-a)^{n-i}$, where $\binom{n}{i} = \frac{n!}{i!(n-i)!}$

When n is prime, $(-a)^{n-1} \equiv 1 \pmod{n}$
so $(-a)^n \equiv -a \pmod{n}$

When n is prime, for 0<i<n,
$\binom{n}{i} \equiv 0 \pmod{n}$



Basic ideas

- Consider polynomial of X
- THM. If a is relatively prime to n, then n is prime if and only if

$$(X-a)^n \equiv X^n - a \pmod{n}$$

- Proof.

$(X-a)^n = \sum_{i=0}^{n}\binom{n}{i}X^i(-a)^{n-i} = X^n + (-a)^n + \sum_{i=1}^{n-1}\binom{n}{i}X^i(-a)^{n-i}$, where $\binom{n}{i} = \frac{n!}{i!(n-i)!}$

When n is composite, let q be a prime factor of n

$$\binom{n}{q} \equiv \frac{n(n-1)\cdots(n-q+1)}{1\cdot 2\cdots q}$$

If k is the largest such that q^k divides n, then q^k does not divide $\binom{n}{q}$

also a^{n-q} is relatively prime to n, so the coefficient of X^q is not zero modulo n



Basic ideas

- Consider polynomial of X
- THM. If a is relatively prime to n, then n is prime if and only if

$$(X-a)^n \equiv X^n - a \pmod{n}$$

- polynomial identity testing, but we want deterministic algorithm!
- Solution: for some small r's, checking

$$(X-a)^n \equiv X^n - a \pmod{X^r - 1, n}$$

  - time: polynomial on r and $\log n$



AKS algorithm

input: $n > 2$
if $n = p^k$ for some prime p, reject;
$r = 2$;
while $(r < n)$ {
    if $(n, r) \neq 1$ reject;
    if r is prime
        $q \leftarrow$the largest prime factor of r-1;
        if $q \geq 4\sqrt{r}\log n$ and $n^{(r-1)/q} \not\equiv 1 \pmod{r}$
            quit the loop with r;
    r++;
}
for $a = 1$ to $2\sqrt{r}\log n$
    if $(X - a)^n \not\equiv X^n - a \pmod{X^r - 1, n}$ reject;
accept;

- Why polynomial time?
- the r we choose has $r = O(\log^6 n)$

THM1. Let P(n) denote the largest prime factor of n, then for large enough x,
$|\{\text{prime } r | r \leq x, P(r-1) > x^{2/3}\}| \geq c \cdot x/\log x$

THM2. The number of primes ≤n is at most 8n/log n



AKS algorithm

input: $n > 2$
if $n = p^k$ for some prime p, reject;
$r = 2$;
while $(r < n)$ {
    if $(n, r) \neq 1$ reject;
    if r is prime
        $q \leftarrow$the largest prime factor of r-1;
        if $q \geq 4\sqrt{r}\log n$ and $n^{(r-1)/q} \not\equiv 1 \pmod{r}$
            quit the loop with r;
    r++;
}
for $a = 1$ to $2\sqrt{r}\log n$
    if $(X-a)^n \not\equiv X^n - a \pmod{X^r - 1, n}$ reject;
accept;

THM1. Let P(n) denote the largest prime factor of n, then for large enough x,
$|\{\text{prime } r | r \leq x, P(r-1) > x^{2/3}\}| \geq 0.6683 \cdot x/\log x$

THM2. The number of primes ≤n is at most 8n/log n

- S is the set of primes in $[c\cdot\log^6 n, 16c\cdot\log^6 n]$ such that $P(r-1) > (16c\cdot\log^6 n)^{2/3}$

$$|S| \geq \frac{0.6683 \cdot 16c \cdot \log^6 n}{7 \log\log n} - \frac{8c \cdot \log^6 n}{6 \log\log n}$$

  which is $\Theta\left(\frac{\log^6 n}{\log\log n}\right)$
- let $x = 16c \cdot \log^6 n$, consider the product

$$\Pi = (n-1)(n^2-1)\ldots(n^{x^{\frac{1}{3}}} - 1)$$

  - this product has $\leq x^{\frac{2}{3}}\log n$ prime factors, which is less than $|S|$
  - so there is a prime r in S which does not divide Π, so r-1 has largest prime factor

$$q > x^{2/3} > 4\sqrt{r}\log n$$

    - since $(r-1)/q < x^{1/3}$,
    - so r does not divide $n^{(r-1)/q} - 1$
- so in $[c\cdot\log^6 n, 16c\cdot\log^6 n]$ we have such an r

Correctness proof -- algebra

- $o_r(p)$: order of p modulo r, smallest positive integer k such that $p^k \equiv 1 \pmod{r}$
- for primes p and r, in field $\mathbf{F}_p$, consider the polynomial:

$$Q_r(X) = \frac{X^r - 1}{X - 1} = X^{r-1} + X^{r-2} + \cdots + X + 1$$

- THM. $Q_r(X)$ factorises into irreducible polynomials each of degree $o_r(p)$ in $\mathbf{F}_p$



Correctness proof -- algebra

- $o_r(p)$: order of p modulo r, smallest positive integer k such that $p^k \equiv 1 \pmod r$
- for primes p and r, in field $\mathbf{F}_p$, consider the polynomial:

$$Q_r(X) = \frac{X^r - 1}{X - 1} = X^{r-1} + X^{r-2} + \cdots + X + 1$$

- THM. $Q_r(X)$ factorises into irreducible polynomials each of degree $o_r(p)$ in $\mathbf{F}_p$

Lemma. Let p be a prime and f(X) be a polynomial with integral coefficients, then $f(X)^p \equiv f(X^p) \pmod p$

Proof. Let $f(X) = a_0 + a_1 X + \cdots + a_k X^k$, then

$$f(X)^p = \sum_{i_0 + \cdots + i_k = p} \frac{p!}{i_0! \ldots i_k!} {a_0}^{i_0} \ldots {a_k}^{i_k} \cdot X^{i_1 + 2i_2 + \cdots + ki_k}$$

$\frac{p!}{i_0! \ldots i_k!}$ is divisible by p unless one of i_j is p

so $f(X)^p = \sum_{0 \le j \le k} {a_j}^p \cdot X^{j \cdot p}$, and ${a_j}^p \equiv a_j \pmod p$



Algebra

Lemma. f(X) be a polynomial with integral coefficients, then $f(X)^p \equiv f(X^p) \pmod{p}$

- $o_r(p)$: order of p modulo r, smallest positive integer k such that $p^k \equiv 1 \pmod{r}$
- for primes p and r, in field $\mathbf{F}_p$, consider the polynomial:

$$Q_r(X) = \frac{X^r - 1}{X - 1} = X^{r-1} + X^{r-2} + \cdots + X + 1$$

- THM. $Q_r(X)$ factorises into irreducible polynomials each of degree $o_r(p)$ in $\mathbf{F}_p$
- Proof.
- Let h(X) be an irreducible factor of $Q_r(X)$ with degree k
- $\mathbf{F}_p[X]/h(X)$ forms a field of size p^k so its multiplicative group is cyclic, let its generator be $g(X)$
- Prove that $k=o_r(p)$:
  - $h(X) \mid (X^r - 1)$ in F_p so $X^r \equiv 1$ in $\mathbf{F}_p[X]/h(X)$, so the order of X in $\mathbf{F}_p[X]/h(X)$ is r
  - $r \mid (p^k - 1)$, and $p^k \equiv 1 \pmod{r}$, so $o_r(p) \mid k$



Algebra

Lemma. f(X) be a polynomial with integral coefficients, then $f(X)^p \equiv f(X^p) \pmod p$

- $o_r(p)$: order of p modulo r, smallest positive integer k such that $p^k \equiv 1 \pmod r$
- for primes p and r, in field $\mathbf{F}_p$, consider the polynomial:

$$Q_r(X) = \frac{X^r-1}{X-1} = X^{r-1} + X^{r-2} + \cdots + X + 1$$

- THM. $Q_r(X)$ factorises into irreducible polynomials each of degree $d=o_r(p)$ in $\mathbf{F}_p$
- Proof.
- Let h(X) be an irreducible factor of $Q_r(X)$ with degree k
- $\mathbf{F}_p[X]/h(X)$ forms a field of size p^k so its multiplicative group is cyclic, let its generator be $g(X)$
- Prove that $k=o_r(p)$: let $d=o_r(p)$, in $\mathbf{F}_p[X]/h(X)$:
  - $g(X)^p \equiv g(X^p) \Rightarrow g(X)^{p^d} \equiv g\left(X^{p^d}\right)$
  - $p^d \equiv 1 \pmod r$, so $p^d = kr+1$, also we have $X^{kr} \equiv 1 \pmod{X^r-1}$, so $X^{p^d} \equiv X \pmod{X^r-1}$
  - Thus $g(X)^{p^d} \equiv g(X) \Rightarrow g(X)^{p^d-1} \equiv 1$
  - The order of $g(X)$ is p^k-1, we get $(p^k-1)|(p^d-1)$, so $k|d$



Correctness

THM. $Q_r(X) = \frac{X^r-1}{X-1}$ factorises into irreducible polynomials each of degree $o_r(p)$ in $\mathbf{F}_p$

input: $n > 2$
if $n = p^k$ for some prime p, reject;
$r = 2$;
while $(r < n)$ {
    if $(n, r) \neq 1$ reject;
    if r is prime
        $q \leftarrow$ the largest prime factor of r-1;
        if $q \geq 4\sqrt{r}\log n$ and $n^{(r-1)/q} \not\equiv 1 \pmod r$
            quit the loop with r;
    r++;
}
for $a = 1$ to $2\sqrt{r}\log n$
    if $(X-a)^n \not\equiv X^n - a \pmod{X^r - 1, n}$ reject;
accept;

- If n is a prime, always accepts
- let n be a composite, if it accepts,

  $\forall\ 1 \leq a \leq \ell = 2\sqrt{r}\log n$, we have

  $(X-a)^n \equiv X^n - a \pmod{X^r - 1, n}$

We know $o_r(n)$ divides r-1
If $o_r(n)$ does not have prime factor q
then $o_r(n)$ divides (r-1)/q, so $n^{(r-1)/q} \equiv 1 \pmod r$
Thus, q satisfies $q | o_r(n)$

- There is a prime factor p of n such that:

  $p^{(r-1)/q} \not\equiv 1 \pmod r$

- Then $q | o_r(p)$
- Next we consider such p



Correctness

THM. $Q_r(X) = \frac{X^r-1}{X-1}$ factorises into irreducible polynomials each of degree $o_r(p)$ in $\mathbf{F}_p$

input: $n > 2$
if $n = p^k$ for som
$r = 2$;
while $(r < n)$ {
    if $(n, r) \neq 1$ reject;
    if r is prime
        $q \leftarrow$the largest prime factor of r-1;
        if $q \geq 4\sqrt{r}\log n$ and $n^{(r-1)/q} \not\equiv 1 \pmod r$
            quit the loop with r;
    r++;
}
for $a = 1$ to $2\sqrt{r}\log n$
    if $(X-a)^n \not\equiv X^n - a \pmod{X^r-1, n}$ reject;
accept;

Also p>r, otherwise we can find p here

- If n is a prime, always accepts
- let n be a composite, if it accepts,

  $\forall\ 1 \leq a \leq \ell = 2\sqrt{r}\log n$, we have

  $(X-a)^n \equiv X^n - a \pmod{X^r-1, n}$

We know $o_r(n)$ divides r-1
If $o_r(n)$ does not have prime factor q
then $o_r(n)$ divides (r-1)/q, so $n^{(r-1)/q} \equiv 1 \pmod r$
Thus, q satisfies $q | o_r(n)$

- There is a prime factor p of n such that:

  $p^{(r-1)/q} \not\equiv 1 \pmod r$

- Then $q | o_r(p)$
- Next we consider such p



Correctness

THM. $Q_r(X) = \frac{X^r-1}{X-1}$ factorises into irreducible polynomials each of degree $o_r(p)$ in $\mathbf{F}_p$

- If the algorithm accepts,

  $\forall\ 1 \le a \le \ell = 2\sqrt{r}\log n$, we have

  $$(X-a)^n \equiv X^n - a \pmod{X^r - 1, n}$$

- let h(X) be the irreducible factor of $Q_r(X)$ of degree $d=o_r(p)$ in $\mathbf{F}_p$
- then $(X-a)^n \equiv X^n - a$ in $\mathbf{F}_p[X]/h(X)$
- Consider the group G in $\mathbf{F}_p[X]/h(X)$:

  $$G = \left\{ \prod_{1\le a\le \ell} (X-a)^{m_a} : m_a \ge 0 \right\}$$

  - G is a subgroup of the cyclic group of field $\mathbf{F}_p[X]/h(X)$, so it is also cyclic
  - Let $S = \{\prod_{1\le a\le \ell}(X-a)^{m_a} : \sum_{1\le a\le \ell} m_a \le d-1,\ m_a \ge 0\}$
  - Then the elements are distinct in G, since a's are distinct modulo p, and their degrees are ≤d-1
  - Also q|d, by $q \ge 4\sqrt{r}\log n$ and $\ell = 2\sqrt{r}\log n$, so $d \ge 2\ell$
- Thus $|G| \ge |S| \ge 2^\ell$



Correctness

THM. $Q_r(X) = \frac{X^r-1}{X-1}$ factorises into irreducible polynomials each of degree $o_r(p)$ in $\mathbf{F}_p$

input: $n > 2$
if $n = p^k$ for some prime p, reject;
$r = 2$;
while $(r < n)$ {
    if $(n, r) \neq 1$ reject;
    if r is prime
        $q \leftarrow$ the largest prime factor of r-1;
        if $q \geq 4\sqrt{r}\log n$ and $n^{(r-1)/q} \not\equiv 1 \pmod r$
            quit the loop with r;
    r++;
}
for $a = 1$ to $2\sqrt{r}\log n$
    if $(X-a)^n \not\equiv X^n - a \pmod{X^r - 1, n}$ reject;
accept;

- If the algorithm accepts,

  $\forall\ 1 \leq a \leq \ell = 2\sqrt{r}\log n$, we have

  $$(X-a)^n \equiv X^n - a \pmod{X^r - 1, n}$$

- let h(X) be the irreducible factor of $Q_r(X)$ of degree $d=o_r(p)$ in $\mathbf{F}_p$
- then $(X-a)^n \equiv X^n - a$ in $\mathbf{F}_p[X]/h(X)$
- Consider the group G in $\mathbf{F}_p[X]/h(X)$:

$$G = \left\{ \prod_{1 \leq a \leq \ell} (X-a)^{m_a} : m_a \geq 0 \right\}$$

  - G is a subgroup of the cyclic group of field $\mathbf{F}_p[X]/h(X)$, so it is also cyclic

- $|G| \geq 2^\ell = n^{2\sqrt{r}}$
- Let $g(X)$ be a generator of G, with order |G|



Some lemmas

- Let $I\big(g(X)\big) = \{m | g(X)^m \equiv g(X^m) \ (mod\ X^r - 1, p)\}$
- Lemma. The set $I\big(g(X)\big)$ is closed under multiplication
- Proof.
  - Let $m_1, m_2 \in I\big(g(X)\big)$, then

    $g(X)^{m_1} \equiv g(X^{m_1}) \ (mod\ X^r - 1, p)$ and $g(X)^{m_2} \equiv g(X^{m_2}) \ (mod\ X^r - 1, p)$
  - we substitute X^{m_1} in place of X in the second congruence:

    $g(X^{m_1})^{m_2} \equiv g(X^{m_1 m_2}) \ (mod\ X^{m_1 r} - 1, p)$
  - which leads to $g(X^{m_1})^{m_2} \equiv g(X^{m_1 m_2}) \ (mod\ X^r - 1, p)$
  - thus

$$g(X)^{m_1 m_2} \equiv g(X^{m_1 m_2}) \ (mod\ X^r - 1, p)$$



Some lemmas

- Let $I\big(g(X)\big) = \{m | g(X)^m \equiv g(X^m) \ (mod\ X^r - 1, p)\}$
- Remind that the order of generator $g(X)$ of G is |G|
- Lemma. Let $m_1, m_2 \in I\big(g(X)\big)$, if $m_1 \equiv m_2 \pmod{r}$, then $m_1 \equiv m_2 \pmod{|G|}$
- Proof.
  - Let $m_2 = m_1 + kr$, then in $\mathbf{F}_p[X]/h(X)$, $g(X)^{m_1+kr} \equiv g\big(X^{m_1+kr}\big)$
  - we have $X^{kr} \equiv 1 \pmod{X^r - 1}$, so $g\big(X^{m_1+kr}\big) \equiv g(X^{m_1})$ in $\mathbf{F}_p[X]/h(X)$
  - So $g(X)^{m_1+kr} \equiv g(X^{m_1}) \equiv g(X)^{m_1}$ and $g(X)^{kr} \equiv 1$ in $\mathbf{F}_p[X]/h(X)$
  - Therefore |G| divides kr and $m_1 \equiv m_2 \pmod{|G|}$

Note $|G| \geq n^{2\sqrt{r}}$ which can be far larger than r



Correctness

Let $I(g(X)) = \{m | g(X)^m \equiv g(X^m) \ (mod\ X^r - 1, p)\}$

Lemma. The set $I(g(X))$ is closed under multiplication

Lemma. Let $m_1, m_2 \in I(g(X))$, if $m_1 \equiv m_2 \pmod{r}$, then $m_1 \equiv m_2 \pmod{|G|}$

- If the algorithm accepts,
- $\forall 1 \le a \le \ell$, $(X-a)^n \equiv X^n - a \pmod{X^r - 1, n}$
- the group $G = \{\prod_{1 \le a \le \ell}(X-a)^{m_a} : m_a \ge 0\}$ in $\mathbf{F}_p[X]/h(X)$ is cyclic and $|G| \ge n^{2\sqrt{r}}$
- Let $g(X)$ be a generator of G, with order |G|
- $g(X)$ also satisfies $g(X)^n \equiv g(X^n) \ (mod\ X^r - 1, p)$
  - so $n \in I(g(X))$
  - since $f(X)^p \equiv f(X^p) \pmod{p}$ for any $f(X)$, so $p \in I(g(X))$
- $I(g(X))$ is closed under multiplication, so $E = \{n^i p^j \mid 0 \le i, j \le \sqrt{r}\} \subseteq I(g(X))$
  - $|E| > r$, so there exists distinct $n^x p^y$ and $n^z p^w$ satisfies $n^x p^y \equiv n^z p^w \pmod{r}$
  - which means $n^x p^y \equiv n^z p^w \pmod{|G|}$, and $n^{x-z} \equiv p^{w-y} \pmod{|G|}$
- Since $|G| \ge n^{2\sqrt{r}}$ and $n^{|x-z|}, p^{|w-y|} < n^{\sqrt{r}}$, we have $n^{x-z} = p^{w-y}$
- It means $n=p^k$ for some k, which cannot pass the first checkpoint



Running time

input: $n > 2$
if $n = p^k$ for some prime p, reject;
$r = 2$;
while $(r < n)$ {
    if $(n, r) \neq 1$ reject;
    if r is prime
        $q \leftarrow$the largest prime factor of r-1;
        if $q \geq 4\sqrt{r}\log n$ and $n^{(r-1)/q} \not\equiv 1 \pmod{r}$
            quit the loop with r;
    r++;
}
for $a = 1$ to $2\sqrt{r}\log n$
    if $(X - a)^n \not\equiv X^n - a \pmod{X^r - 1, n}$ reject;
accept;

find q takes $\sqrt{r}$ time

- r is bounded by $O(\log^6 n)$

while loop takes $\tilde{O}(\log^9 n)$



Running time

input: $n > 2$
if $n = p^k$ for some prime p, reject;
$r = 2$;
while $(r < n)$ {
    if $(n, r) \neq 1$ reject;
    if r is prime
        $q \leftarrow$the largest prime factor of r-1;
        if $q \geq 4\sqrt{r}\log n$ and $n^{(r-1)/q} \not\equiv 1 \pmod{r}$
            quit the loop with r;
    r++;
}
for $a = 1$ to $2\sqrt{r}\log n$
    if $(X - a)^n \not\equiv X^n - a \pmod{X^r - 1, n}$ reject;
accept;

- r is bounded by $O(\log^6 n)$
- So total time is $\tilde{O}(r^{3/2} \cdot \log^3 n) = \tilde{O}(\log^{12} n)$
  - later improved to $\tilde{O}(\log^6 n)$ by Lenstra and Pomerance

- a bounded by $O(\sqrt{r}\log n)$
- n^{th} power takes $O(\log n)$ time
- every polynomial operation takes $\tilde{O}(r \cdot \log n)$ time

